BD 2 - Autenticazione con Flask

Alessandra Raffaetà

Università Ca' Foscari Venezia



Università
Ca'Foscari
Venezia

Introduzione

In questa lezione mettiamo insieme i pezzi studiati nelle ultime settimane, sviluppando un sistema di autenticazione basato su Flask.

Ingredienti:

- conoscenza base di Python e Flask
- un database di utenti, con cui ci interfacceremo via SQLAlchemy
- una libreria di autenticazione (Flask-Login)
- una libreria per l'hashing delle password (Passlib)

Quanto studieremo oggi potrà essere esteso per diventare il primo pezzo del vostro progetto.

Database di Utenti

Memorizziamo gli utenti in una tabella `users` con attributi:

- `id`: int chiave primaria
- `username`: username dell'account (UNIQUE)
- `password`: password di accesso dell'account (si salva un suo *hash*)

Esercizio: Scrivere uno script Python che:

- si connette al vostro DBMS preferito tramite SQLAlchemyORM
- crea la classe `User` che modella la tabella users con la struttura indicata
- inserisce due utenti di prova (Alice e Bob)

Flask-Login

Flask-Login è una libreria di autenticazione allo stato dell'arte per Flask:

- implementa l'autenticazione tramite le sessioni native di Flask
- consente di limitare l'accesso a certe pagine ai soli utenti autenticati
- indipendente dalla scelta del DBMS usato per gli utenti: in effetti potreste benissimo salvare gli utenti in un file esterno!
- non supporta autorizzazione: è vostro compito implementarla, ma sono disponibili altre estensioni di Flask (per esempio Flask-RBAC) per tale scopo

Installabile tramite: `pip install flask-login`

Flask-Login: Configurazione

L'inizializzazione di Flask-Login richiede una chiave segreta, che viene utilizzata per firmare un cookie da impostare nel browser degli utenti:

```
from flask_login import LoginManager, UserMixin, login_user,
                        logout_user, current_user, login_required

app = Flask(__name__)
app.config['SECRET_KEY'] = 'ubersecret'

login_manager = LoginManager()
login_manager.init_app(app)
```

Il cookie salva nel client le informazioni della sessione autenticata, fra cui l'identità dell'utente autenticato.

Flask-Login: Utilizzo

Flask-Login si appoggia ai seguenti ingredienti principali:

1. una classe utente che definisce la struttura degli utenti del sistema, che deve includere un attributo univoco detto identità (tipo: stringa)
2. una funzione `login_user(usr)`, che salva nella sessione l'identità dell'utente autenticato, eseguendo quindi il login
3. una funzione `logout_user()`, che fa l'opposto (logout)
4. una callback che trasforma l'identità precedentemente salvata nella sessione in un'istanza della classe utente, che viene resa accessibile a Flask tramite l'oggetto `current_user`
5. un decoratore `@login_required` per proteggere le route che richiedono autenticazione

E' compito del programmatore definire 1 e 4.

Struttura degli Utenti

La classe `User` eredita dalla classe `UserMixin` di Flask-Login. E' usata per portare le informazioni sugli utenti dal database a Flask:

```
class Base(DeclarativeBase):
  pass

db = SQLAlchemy(model_class=Base)

class User(UserMixin, db.Model):
    __tablename__ = "users"

    id: Mapped[int] = mapped_column(primary_key=True)
    username: Mapped[str] = mapped_column(unique=True, nullable=False)
    password: Mapped[str] = mapped_column(nullable=False)
```

È richiesta la definizione di un attributo di nome `id`, a meno che non vogliate definirvi un metodo `get_id()` appropriato.

Callback

La callback `load_user` ha il compito di trasformare un'identità (stringa) in un'istanza della classe `User`, si noti l'uso del decoratore:

```
@login_manager.user_loader
def load_user(user_id):
    return db.session.get(User, int(user_id))
```

Struttura degli Utenti

Oltre a ciò che definite voi, ciascuna classe utente mette nativamente a disposizione i seguenti attributi e metodi via ereditarietà da `UserMixin`:

- `is_authenticated`: attivato se l'utente ha presentato credenziali valide, è l'attributo controllato dal decoratore `@login_required`
- `is_active`: può essere disattivato per bloccare un account
- `is_anonymous`: attivato solo per gli utenti non autenticati
- `get_id()`: ritorna l'identità dell'utente

I cambiamenti dell'attributo `is_authenticated` sono effettuati tramite invocazione degli appropriati metodi di login / logout di Flask-Login.

Implementare la Home

La home ha il compito di rimandare gli utenti autenticati all'area riservata. Gli utenti anonimi sono invece invitati a fare login:

```
@app.route('/')
def home():
    # current_user identifica l'utente attuale
    # utente anonimo prima dell'autenticazione
    if current_user.is_authenticated:
        return redirect(url_for('profile'))

    num_utenti = db.session.scalar(db.select(func.count()).
                                   select_from(User))
    return render_template("home.html", valore = num_utenti)
```

Implementare l'Area Riservata

L'area riservata è dedicata ai soli utenti autenticati e visualizza le informazioni relative all'utente autenticato:

```
@app.route('/profile')
@login_required
def profile():
    return render_template("profile.html", user = current_user)
```

Le view accessibili solo agli utenti autenticati possono essere decorate con `@login_required`.

Implementare il Login

```
@app.route('/login', methods = ['GET', 'POST'])
def login():
  if request.method == 'POST':
    username = request.form["username"]
    password = request.form["password"]

    user = db.session.scalars(db.select(User).
          filter_by(username = username)).one_or_none()

    if user is not None and bcrypt.verify(password, user.password)
      login_user(user)
      return redirect(url_for('profile'))

    flash("ERRORE Credenziali!")
    return redirect(url_for('home'))
  return redirect(url_for('home'))
```

dove la funzione `load_user` ritorna un'istanza della classe `User` a partire dall'username dell'utente (definita in precedenza).

Implementare il Logout

L'implementazione del logout è molto semplice:

```
@app.route('/logout')
@login_required
def logout():
    flash("Sconnesso!")
    logout_user()
    return redirect(url_for('home'))
```

L'effetto della chiamata è la rimozione dell'identità dell'utente dalla sessione corrente.

Memorizzazione Sicura delle Password

- Memorizzare le password in chiaro nel database rappresenta un grave rischio per la sicurezza.
- Le password devono essere sempre sottoposte a:
  - **Hashing**: trasformazione irreversibile della password;
  - **Salting**: aggiunta di un valore casuale, salt, prima del calcolo dell'hash.
- Nel database viene memorizzato l'hash della password, non la password originale.
- Durante il login:
  1. si recupera l'hash memorizzato;
  2. si applica alla password fornita dall'utente lo stesso algoritmo di hashing;
  3. si confrontano i due hash.

Passlib e bcrypt

- `bcrypt` è un algoritmo di hashing progettato per la protezione delle password.
- `Passlib` è una libreria Python che fornisce un'interfaccia semplice e uniforme per diversi algoritmi di hashing.
- Nel nostro progetto utilizziamo `Passlib` con l'algoritmo `bcrypt`.
- Per installare `Passlib` usare: `pip install passlib`

```
from passlib.hash import bcrypt

# Per memorizzare la password
hashed_password = bcrypt.hash(password)

# Per verificare la password
if bcrypt.verify(password, hashed_password):
    print("Password corretta")
```

Gestione del Salt con bcrypt

- `bcrypt` genera automaticamente un **salt casuale e univoco** per ogni password.
- Il salt viene incorporato direttamente nella stringa contenente l'hash della password.
- Non è quindi necessario:
  - memorizzare il salt in una colonna separata del database;
  - gestire manualmente la generazione dei salt.
- Durante la verifica delle credenziali:
  1. `bcrypt.verify()` estrae automaticamente il salt dall'hash memorizzato;
  2. applica lo stesso algoritmo alla password fornita dall'utente;
  3. confronta il nuovo hash con quello salvato nel database.

Messaggi Temporanei con `flash()`

- Una buona applicazione web deve fornire feedback alle azioni dell'utente.
- Flask mette a disposizione il meccanismo di message flashing, che consente di memorizzare un messaggio al termine di una richiesta HTTP e visualizzarlo nella richiesta successiva.
- I messaggi flash sono particolarmente utili per:
  - confermare operazioni completate con successo;
  - segnalare errori di autenticazione;
  - notificare eventi importanti all'utente.
- Il messaggio viene normalmente visualizzato all'interno di un template comune (ad esempio `base.html`) e rimane disponibile per una sola richiesta.
- inserire: `from flask import flash`

Esempio di Template per messaggi `flash()`

Base.html

```
<html>
<head>
  <title>Flask demo</title>
  <link rel="stylesheet" href="/static/style.css">
</head>
<body>
{% with messages = get_flashed_messages() %}
  {% if messages %}
    <ul class=flashes>
    {% for message in messages %}
      <li>{{ message }}</li>
    {% endfor %}
    </ul>
  {% endif %}
{% endwith %}
{% block pagebody %}{% endblock %}
</body>
</html>
```

Checkpoint!

Provate ora a ricostruire l'applicazione che abbiamo discusso in queste slides sulla vostra macchina! In particolare, dovreste avere:

- un'applicazione Flask con 4 route: una per la pagina di benvenuto, una per il login, una per l'area riservata ed una per il logout
- una directory con 2 template: uno per la pagina di benvenuto (login form) ed uno per l'area riservata (Informazioni personali e pulsante di logout)

Riutilizzate il codice presentato a lezione e completate i pezzi mancanti, in particolare l'integrazione col DBMS, per ottenere una prima web app funzionante.